

2026 SPS Modelling Competition

Saturday, October 3, 2026

Problem Set and Submission Guide

Competition at a Glance

Location	TS3 0270, George/Scheller Tower	Program start	9:00 AM
Prompts released	10:00 AM	Submission deadline	10:00 PM
Team size	1–4; solo participants welcome	Prize	\$200 to the winning team for each prompt
Results	Livestreamed Sunday morning	Prerequisites	None! This is intended as a learning experience

General Instructions

Choose **one of the three prompts** in this packet. There is no single intended method. State your assumptions, explain why you chose your model, test the parts of it that can be checked, and tell us what you learned from it.

You may use programming languages, numerical and symbolic packages, public datasets, papers, textbooks, and AI tools. Cite outside sources. If the prompt does not specify a quantity, choose a value or distribution you can defend and check whether changing it changes your conclusion.

What to Submit by 10:00 PM

Send one email to spsgatech@gmail.com containing or linking to all of the following:

- a **5–7 minute recorded presentation**; **7:00 is a hard maximum**;
- either a **slide deck or report** that explains the model, assumptions, calculations, results, and limitations in enough detail for us to follow your work;
- all **code and data** needed to reproduce the principal results, with a short README or one obvious entry point; and
- an **AI-use statement** in the slides/report. If you used AI, name the tool(s), say what you used them for, give one concrete example, and explain how you checked the result. Also make clear which modelling, derivations, coding, or interpretation were done by your team. A full chat transcript is not required. If you did not use AI, say so.

Use the video to explain your actual process. We want to see how you chose the model, what you tried, what changed, what the model predicts, and how you checked it. If you used AI, explain how it saved time or helped with a specific task and how you checked its output. Do not use an AI response as a citation.

Logistics and Help

Food will not be provided, so plan accordingly; there are food options around George Tower in Midtown. A support desk will be staffed in the competition room for four hours after prompt release. After that, questions may be posted in the [competition Discord](#). General questions may be sent to sps.gatech@gmail.com; additional information is available at spsmodelling.web.app. Participants who ordered a T-shirt may pick it up at the competition; additional shirts will be available for purchase while supplies last.

How Judging Works

Most of the score comes from the technical work. The remaining points are for explaining the work clearly enough that we can judge it and, where applicable, reproduce it. Each prompt has **80 prompt-specific technical points** and the same **20 communication and reproducibility points** below.

Common judging criterion	Points
Recorded presentation: explains the approach and reasoning, uses figures or a demo effectively, interprets the results correctly, and stays within 7 minutes	10
Slides/report: readable technical explanation with labelled figures/tables, citations, assumptions, and enough detail to check the main calculations	5
Reproducibility: runnable code, documented inputs/outputs, fixed seeds where appropriate, and results that can be regenerated	3
AI-use disclosure: clear description of what AI was used for and how its output was checked	2
Common subtotal	20

What we will look for

Across all three prompts, we will look for the following:

1. **Formulate.** Define the question, state variables, units, assumptions, and objective.
2. **Build.** Derive or motivate the model; a single library call is not a model.
3. **Check the implementation.** Use conservation laws, limiting cases, hand calculations, grid/step refinement, or other known results when they apply.
4. **Compare.** Use data, a simpler baseline, a held-out case, or an independent calculation.
5. **Test assumptions.** Change important parameters or modelling choices.
6. **Interpret.** Explain what the result means physically, biologically, or algorithmically.

Time management. You have twelve hours from prompt release to the deadline. Get a baseline working before adding complexity. A simple model that is correct, tested, and well-explained can score much better than a complicated but unfinished model.

Contents

1	Routing on Curved Worlds	3
2	Quantum Hardware Architecture	7
3	Differential-Equation Modelling	9

Prompt 1

Routing on Curved Worlds

Route planning and resource allocation on a disk, sphere, flat torus, and flat Klein bottle.

A food distributor must serve 50 destinations on each of four worlds. Each world has a depot, 50 checkpoints, 1–3 ocean regions, and destinations with different demands. Some destinations are islands. You have a limited token budget and three transport modes. Your goal is to design an algorithm that produces high-quality deliveries across all four geometries.

The supplied `optimal_transport.py` defines the public instance, geometry, feasibility checks, and score. It uses seed 67 for the public instance. You may import it, but do not change the reference constants or scoring functions when reporting a score. We may also run your method on other seeds, so do not hard-code the public instance.

Geometry

All distances are in meters and one model time unit is one day. In this prompt, a *metric* is simply the rule used to measure distance, and a *geodesic* is a shortest route according to that rule.

World	Scale	Distance model
Disk	radius 1,000 km	Euclidean distance inside the filled disk
Sphere	radius 1,000 km	great-circle distance
Flat torus	square side 2,000 km	shortest Euclidean displacement over periodic copies
Flat Klein bottle	square side 2,000 km	shortest Euclidean displacement among equivalent lifts under the Klein identifications

For the flat torus, $(x, y) \sim (x + L, y) \sim (x, y + L)$. For the flat Klein bottle, the reference code uses

$$(x, y) \sim (x + L, y), \quad (x, y) \sim (-x, y + L).$$

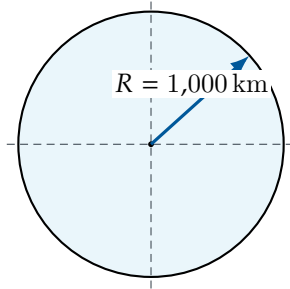
For the torus and Klein bottle, the square is a fundamental domain: coordinates related by the edge identifications represent the same point. The drawings on the next page are only pictures of the topology. **Compute distances using the flat quotient rules above, not straight-line distances in the 3D drawings.**

Geometry background. You do not need a differential geometry course for this prompt. If the words metric or geodesic are unfamiliar, a useful free reference is Theodore Shifrin's [Differential Geometry: A First Course in Curves and Surfaces](#). The sections on the first fundamental form and geodesics give more background than you need here.

Island and ocean rule. Horses and trucks cannot cross an ocean region and cannot serve an endpoint marked as an island. The island flag in the generated instance is authoritative even if a plot looks ambiguous. Jets may cross oceans and serve islands. For official scores, the reference function `leg_crosses_ocean` is authoritative for ground routes. It samples each leg at 96 points, so use that function rather than substituting a different intersection test when reporting an official score.

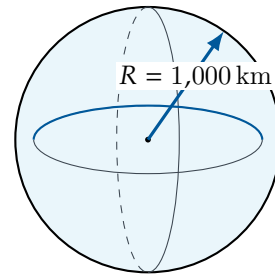
Geometry Reference for Prompt 1

Flat disk



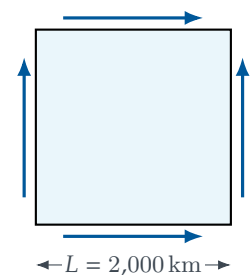
Distance is ordinary Euclidean distance inside the disk.

Sphere

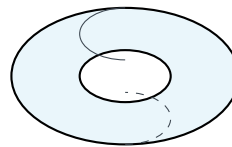


Distance is the length of the shorter great-circle arc.

Flat torus



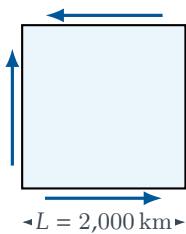
A. fundamental square



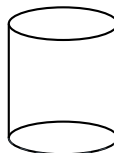
B. topological picture

Identify the two pairs of opposite edges with matching arrow directions. Route lengths are computed in the square using periodic copies. The doughnut in B is only a picture of the topology.

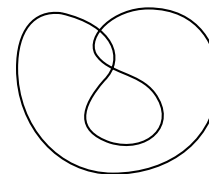
Flat Klein bottle



A. fundamental square



B. cylinder



C. Klein-bottle sketch

A $\rightarrow$ B: identify the left and right sides, whose arrows point the same way, to make a cylinder. B $\rightarrow$ C: the top and bottom sides of the original square become the two circular ends of the cylinder; identify those ends with a reversal. The crossing in C only appears because a Klein bottle cannot be embedded in three-dimensional space without self-intersection.

Figure note. These topology schematics were generated with ChatGPT and then checked and edited by the organizers so that the arrow identifications agree with `optimal_transport.py`. The equations in the prompt and the starter code are the reference definitions if a drawing is unclear.

Resource model and score

You have **1000 tokens per world**. The dispatch cost in the table is charged once for every feasible delivery in the submitted plan.

Mode	Dispatch tokens	Fuel/unit tokens	Mileage km/unit	Speed km/day	Capacity kg	Max leg km
Horse	25	5	100	50	100	50
Truck	100	10	300	500	1200	750
Jet	300	100	1500	5000	5000	3000

A checkpoint adds one day. A route leg longer than the mode's maximum leg is invalid. If a route has total length D , speed v , and n_{cp} checkpoint stops, then

$$t = \frac{D}{v} + n_{cp}.$$

Fuel is continuous. If mileage is m and fuel price is c_f tokens per unit,

$$C_{\text{fuel}} = c_f \frac{D}{m}.$$

Let D_0 be the direct geodesic distance and $\rho = D/D_0 \geq 1$. The attack probability and wear factor are

$$p_{\text{attack}} = 1 - \exp[-0.10 t e^{-3(\rho-1)}], \quad W(\rho) = e^{-0.20(\rho-1)}.$$

An attack removes 25% of the load in expectation. Food is multiplied by $(0.95)^t$ and the nutrition factor is $e^{-0.02t}$.

For endpoint j , let L_j be the load sent and D_j its demand. The exact quality term used by the scorer is

$$q_j = \min\left(\frac{L_j}{D_j}, 1\right) (0.95)^t e^{-0.02t} (1 - 0.25 p_{\text{attack}}) W(\rho).$$

Sending more than the endpoint demand does not increase q_j . In particular, sending extra food does not compensate for decay because the served fraction is capped before the decay factors are applied.

If you submit more than one feasible delivery to the same endpoint, only the largest q_j is used, but the token cost of every feasible delivery still counts. Invalid deliveries contribute no quality and are ignored by the token sum in the reference scorer.

For one world,

$$Q = \frac{\sum_j D_j q_j}{\sum_j D_j}.$$

If total dispatch plus fuel cost exceeds 1000 tokens, that world's score is zero. Otherwise,

$$C_M = 100Q \left[0.85 + 0.15 \left(1 - \min\left(\frac{F_{\text{tok}}}{1000}, 1\right) \right) \right].$$

The four-world score is

$$C = \frac{C_{\text{disk}} + 2C_{\text{sphere}} + 3C_{\text{torus}} + 5C_{\text{Klein}}}{11}.$$

Required grading interface

Your repository must contain a file named `solve.py` at the top level. We will run

```
python solve.py --seed N --out plan.json
```

for the public seed and a few organizer seeds. If your optimizer is written in another language, `solve.py` may call it. The judging run will not assume network access. If you need extra packages, include installation instructions or an environment file. Setup time is not part of the runtime limit. The time limit is **120 seconds per seed**.

The output file must be a JSON object with one list of deliveries per world. A minimal example is

```
{"disk":[...], "sphere":[], "torus":[...], "klein":[]}
```

A delivery is a JSON object such as "endpoint": 3, "vehicle": "truck", "checkpoints": [12]. It may also contain "load_kg": If load is omitted, the scorer uses the destination demand. Test a plan with

```
python optimal_transport.py --seed N --score-plan plan.json.
```

A missing world counts as an empty list. If `solve.py` fails, times out, or writes invalid JSON, the performance score for that seed is zero. The rest of the project is still graded normally. Organizer seeds will be screened so that the depot is not inside an ocean on any world.

For each organizer seed we compute the four-world score C . If H is your mean C over the organizer seeds and H_{best} is the largest valid mean among submitted teams, the performance part of the rubric is

$$P_{\text{perf}} = 20 \min\left(1, \frac{H}{H_{\text{best}}}\right).$$

Your challenge

Part I - Get the geometry and a baseline working

Implement or use the distance functions on all four worlds and build at least one simple feasible baseline, such as cheapest feasible direct delivery, greedy benefit-per-token, or a shortest-path/checkpoint heuristic. Before optimizing, include the following short checks so that we can tell your geometry code is doing what you think it is doing:

- **Disk:** compare one computed distance with the ordinary Euclidean distance you calculate by hand.
- **Sphere:** use a simple case with a known answer, for example two points separated by 90° on a great circle, whose distance is $\pi R/2$.
- **Torus:** choose points near opposite sides of the square and show that wrapping across the boundary gives the shorter route. Also check that shifting a point by L does not change the distance.
- **Klein bottle:** choose a point and one of its equivalent copies under the identifications above and check that the distance between them is zero. For one nontrivial pair, compare the code with the shortest distance you get by checking several equivalent copies by hand.

Also check a few point pairs in each geometry to make sure reversing the endpoints does not change the distance: $d(A, B) = d(B, A)$.

Part II - Beat your baseline

Design an optimization strategy that chooses endpoints, modes, loads, and checkpoint sequences. Exact optimization is not required. Greedy methods, graph search, dynamic programming, mixed-integer optimization, local search, evolutionary methods, or hybrids are all acceptable if you can explain why they fit the problem.

Before building a large search, inspect the constants. You do not get extra credit for using every vehicle or every checkpoint. If a mode, load choice, or route family is never useful under the reference rules, show why and remove it from the search. That simplification is part of the solution. If you evaluate many candidate routes, it will usually be faster to precompute their feasibility and quality terms instead of repeatedly calling the full scorer inside an inner loop.

Report your score on seed 67 and compare it directly to your baseline.

Part III - Show that it generalizes

Test at least two additional seeds or change at least two important parameters. Show one case where the method changes behavior or fails. Explain what you would change if you had more time. When you improve the score on seed 67, check that the same idea also helps on at least one other instance.

Technical rubric (80 points)

Correct geometry, edge identifications, ocean/island rules, and the checks above	15
Performance on the organizer seeds, using the formula above	20
Optimization method and comparison with a simple baseline	20
Useful simplifications: identifying dominated choices or reducing the search without changing the score	10
Generalization / sensitivity tests on other seeds or parameters	10
Interpretation of the result and main limitations	5
Technical subtotal	80

Prompt 2

Quantum Hardware Architecture

Choose a less-established qubit platform, design an architecture around it, and determine whether it could scale.

Choose a quantum-computing modality that is not one of the standard primary platforms for this competition: **do not use trapped ions, neutral atoms, superconducting circuits, or photonic qubits as the primary qubit**. A mature platform may appear as an auxiliary control, memory, or interconnect. Semiconductor spin qubits and NV centres are allowed only if your proposal is substantially different from a standard design.

Possible directions include electrons on helium or solid neon, molecular electron/nuclear-spin qubits, less-common colour centres, excitonic or valley degrees of freedom, and magnonic or phononic encodings. You may choose something else if you can support it with literature.

Keep experimental facts separate from your extrapolations. Mark important numbers as *measured*, *simulated*, or *assumed*. Use primary papers for experimental values when possible. At least two technical references should be primary research papers.

The DiVincenzo criteria

For this prompt, use the original physical-implementation framework as a checklist. A computing platform should provide:

1. a scalable physical system with well-characterized qubits;
2. the ability to initialize the qubits to a simple fiducial state;
3. decoherence times long compared with gate-operation times;
4. a universal set of quantum gates;
5. qubit-specific measurement;
6. for quantum communication or modular architectures, interconversion between stationary and flying qubits; and
7. faithful transmission of flying qubits between specified locations.

If your architecture is monolithic, criteria 6–7 may be treated as an extension, but you must still discuss how modules would communicate if a single device cannot scale indefinitely.

Your challenge

Part I - Build an evidence audit

Define the computational basis and write down the relevant Hamiltonian or level structure at the level needed to explain control. Create a compact evidence table using the quantities that apply to your platform: operating temper-

ature, qubit splitting, initialization method and timescale, T_1 , T_2 or analogue, one- and two-qubit gate mechanism, gate time/fidelity, measurement method/fidelity, connectivity, physical pitch/density, and fabrication/trapping requirements. Mark every entry as **measured**, **simulated**, or **your estimate**.

Part II - Design the architecture

Propose a processor architecture and evaluate it against the DiVincenzo criteria. Include a diagram showing qubits, controls, readout, couplers/interconnects, and the scaling unit (chip, trap, molecule, module, etc.). Address:

- initialization, coherent single-qubit control, an entangling interaction, and readout;
- connectivity and routing of quantum information;
- dominant decoherence, leakage, cross-talk, fabrication, control-bandwidth, vacuum/cooling, and calibration limits;
- how the device grows from a few qubits to at least one useful error-correcting block; and
- what hardware must be replicated per qubit versus shared across a module.

Part III - Quantify the main bottleneck

Do at least **one back-of-the-envelope or numerical calculation of your own**. Do not copy the final number from a paper. You might estimate a coherence-to-gate-time budget, two-qubit gate time from a coupling strength, control-line density, cryogenic heat load, fabrication yield versus device size, optical collection efficiency, error-correction overhead, or module-to-module communication rate.

Compare your modality with **two established platforms** using the same metrics. Then answer two questions directly: What is the most serious physical or engineering reason this architecture might fail to scale? What experiment would give you the clearest answer?

Fault tolerance. You do not need a complete fault-tolerant design. Relate the physical error mechanisms you expect to an error-correcting code or strategy and estimate what physical performance it would require. If the platform has no measured two-qubit gate, state that clearly and use a required target instead of inventing a measurement.

Technical rubric (80 points)

Physical qubit definition and quality of the evidence audit; measured vs. assumed quantities are kept distinct	20
Architecture and treatment of the relevant DiVincenzo criteria, including control, readout, connectivity, and scaling	20
Original quantitative resource / bottleneck analysis with defensible assumptions	20
Noise, fault-tolerance path, and quantitative comparison with two established platforms	15
Originality and a clear account of the main scaling risk and next experiment	5
Technical subtotal	80

Suggested starting references

These are starting points, not required sources and not substitutes for searching the literature relevant to your chosen modality.

- D. P. DiVincenzo, “The Physical Implementation of Quantum Computation,” *Fortschritte der Physik* 48, 771–783 (2000), arXiv:quant-ph/0002077.
- E. Kawakami *et al.*, “Blueprint for quantum computing using electrons on helium,” *Physical Review Applied* 20, 054022 (2023).
- M. R. Wasielewski *et al.*, “Exploiting chemistry and molecular systems for quantum information science,” *Nature Reviews Chemistry* 4, 490–504 (2020).

- S. L. Cornish, M. R. Tarbutt, and K. R. A. Hazzard, “Quantum computation and quantum simulation with ultracold molecules,” *Nature Physics* 20, 730–740 (2024).

Prompt 3

Differential-Equation Modelling

Build, test, and interpret a dynamical model of traffic or an interacting ecological network.

Choose **one** of the two tracks below. Ordinary, partial, delay, stochastic, or integro-differential equations are all allowed if you explain why they are useful. You may use public data. If you use synthetic data, state the model that generated it, add a reasonable noise/error model, and keep a time interval or perturbation case separate for testing.

Check your solver. A smooth-looking plot can still come from an incorrect numerical method. Include at least one check using conservation, step-size or grid refinement, an analytic/limiting solution, positivity, or another property your model should satisfy.

Track A - Traffic flow after a disruption

A natural continuum baseline is the Lighthill–Whitham–Richards conservation law

$$\frac{\partial \rho}{\partial t} + \frac{\partial q(\rho)}{\partial x} = 0,$$

with a chosen fundamental diagram $q(\rho)$. The cell-transmission model is a convenient finite-volume/discrete analogue. Car-following ODEs or another defensible model are also acceptable.

Part I - Baseline and verification

Define the road geometry, density/flow variables, units, boundary conditions, and the fundamental diagram or microscopic rules. Show that the implementation conserves vehicles to numerical tolerance. If you use a finite-volume or finite-difference method, state the **Courant–Friedrichs–Lewy (CFL) condition** you use. For a standard first-order LWR scheme, a typical condition is $\max_{\rho} |q'(\rho)| \Delta t / \Delta x \leq 1$. If you use a different numerical method, give the corresponding stability criterion.

Part II - Incident model and prediction

Introduce a time-dependent disruption such as a lane closure, reduced capacity, traffic light, changing inflow, or reopening. Predict at least two operational quantities, for example queue length, shock position, delay, throughput, travel time, or recovery time. Calibrate parameters from data or explain how your synthetic values were selected.

Part III - Extend and test the model

Add one extension that changes a quantity you care about, for example a network junction, second vehicle class, second-order dynamics, stochastic demand, or a control policy. Compare it with the baseline on a case you did not use while tuning the model. Change at least three important parameters or modelling choices and note which conclusions remain the same.

Track B - Food-web dynamics and resilience

A generalized Lotka–Volterra model is a reasonable baseline for $n \geq 5$ interacting populations,

$$\dot{x}_i = x_i \left(r_i + \sum_{j=1}^n A_{ij} x_j \right),$$

but you may instead motivate nonlinear functional responses, seasonal forcing, harvesting, migration, stochasticity, resource limitation, or another ecological mechanism.

Part I - Baseline and local stability

Specify the species or functional groups, interaction signs, units, parameter ranges, and initial conditions. Find or numerically identify an equilibrium and analyze its local stability with the Jacobian/eigenvalues when that analysis applies. Check that populations stay nonnegative and that your numerical conclusions do not change under a reasonable change in time step.

Part II - Perturb the ecosystem

Study a concrete dynamical question: recovery after a population crash, harvesting pressure, trophic cascade, coexistence, extinction risk, invasion, seasonal forcing, or an environmental shift. Define a quantitative resilience/stability measure and use the model to make a prediction.

Part III - Extend and test the model

Compare the baseline with at least one richer interaction model, such as a Holling-type saturating response, predator interference, carrying capacities, or structured network effects. Test it on a perturbation or dataset you did not use when choosing parameters. Perform sensitivity analysis on at least three influential parameters. You may also connect the result to the classic complexity–stability question if that comparison helps explain your food web.

Common requirements for either track

- 1. Define state variables, units, parameters, initial/boundary conditions, and the numerical method.
- 2. Explain how parameters were calibrated or selected and distinguish fitted parameters from assumptions.
- 3. Compare a simple baseline model with at least one justified improvement.
- 4. Test the model on held-out data, a new perturbation, or an independent calculation.
- 5. Change at least three parameters or modelling choices and report how the conclusion changes.
- 6. Explain the successes and failures in physical or biological terms. Do not rely on one error metric alone.

Technical rubric (80 points)

Model formulation, assumptions, units, and physical/biological justification	20
Numerical implementation and verification: conservation, stability, convergence, positivity, or equivalent checks	15
Calibration / parameter selection and out-of-sample validation	15
Quality of the disruption/perturbation prediction and evidence supporting it	15
Sensitivity, uncertainty, and failure-mode analysis	10
Insight from the model improvement and interpretation of what changed	5
Technical subtotal	80

Suggested starting references

- C. F. Daganzo, “The Cell Transmission Model: A Dynamic Representation of Highway Traffic Consistent with the Hydrodynamic Theory,” *Transportation Research Part B* 28, 269–287 (1994).
- R. M. May, “Will a Large Complex System be Stable?,” *Nature* 238, 413–414 (1972).
- S. Allesina and S. Tang, “Stability criteria for complex ecosystems,” *Nature* 483, 205–208 (2012).
- For nonlinear predator–prey interactions, search the literature on Holling type-II / type-III and predator-dependent functional responses; justify whichever form you use.

Final reminder. The deadline is **10:00 PM**. Your final submission should make it possible for a judge to answer four questions quickly: What did you model? Why is the model reasonable? What did it predict? Why should we trust the result?